

EXAMEN FINAL — SEMESTRE 2 (2025/2026)

Module : **Base de Données Avancée**

Filière : Génie Informatique — CI1-G1

SIEM

Security Information and Event Management

Conception et programmation avancée d'une base de données
de centralisation des alertes de cybersécurité

PostgreSQL • MongoDB • Python

Étudiant : Yassir ZAHIDI

Encadrant : Pr. Achraf Chakir BARAKA

Classe : CI1 G1 — Hay Riad

Table des matières

1	Introduction	1
1.1	Présentation du système choisi	1
1.2	Objectifs du projet	1
1.3	Architecture générale	2
2	Partie I — Base de données relationnelle (PostgreSQL)	3
2.1	Description des tables créées	3
2.1.1	Modèle Conceptuel de Données (MCD)	3
2.1.2	Liens et règles d'intégrité	4
2.2	Script SQL complet	4
2.2.1	Création des séquences	4
2.2.2	Création des tables principales	5
2.2.3	Fonctions PL/pgSQL	6
2.2.4	Triggers avec règles de gestion	7
2.2.5	Procédures stockées avec boucles	8
2.2.6	Vue globale combinant les tables	9
2.3	Explication des choix techniques	10
2.3.1	Types de données avancés	10
2.3.2	Index et performance	10
2.3.3	Sécurité — pgcrypto	10
3	Partie II — Base NoSQL (MongoDB)	12
3.1	Description de la collection	12
3.2	Justification des champs (au moins 15)	12
3.3	Insertion des données	13
3.4	Requêtes réalisées	13
3.4.1	Requête avec condition numérique	13
3.4.2	Requête avec plusieurs conditions (AND/OR)	14
3.4.3	Recherche textuelle	14
3.4.4	Mise à jour	14
3.4.5	Suppression	14
3.4.6	Agrégations (moyenne, somme, groupement)	15
3.4.7	Projection	15
3.4.8	Index Mongo	16
3.5	Visualisation des données	16
4	Conclusion	18
4.1	Synthèse du travail réalisé	18
4.2	Difficultés rencontrées	18

4.3 Perspectives d'amélioration	19
---	----

1. Introduction

1.1 Présentation du système choisi

Le projet consiste en la conception et l'implémentation d'un **SIEM** (*Security Information and Event Management*), c'est-à-dire un système de centralisation et d'analyse des alertes de sécurité informatique. Ce type de plateforme est utilisé quotidiennement par les équipes SOC (Security Operations Center) des grandes entreprises (banques, opérateurs télécoms, hôpitaux) pour détecter, qualifier et traiter les attaques informatiques.

Le système surveille un parc d'équipements informatiques (serveurs, firewalls, routeurs, postes utilisateurs) et reçoit en continu des alertes émises par des règles de détection (signatures d'attaques connues : force brute, injection SQL, déni de service, etc.). Chaque alerte est qualifiée automatiquement par un score de risque (0–100) et, si le volume d'alertes critiques dépasse un seuil sur un même équipement, un *incident* agrégé est créé automatiquement et assigné à un analyste SOC.

Note technique

Pourquoi ce sujet ? La cybersécurité est aujourd'hui un enjeu stratégique au Maroc (loi 09-08 sur la protection des données, Bank Al-Maghrib, DGSSI). Un SIEM est l'outil de référence dans ce domaine et illustre parfaitement la complémentarité **PostgreSQL** (intégrité transactionnelle des incidents) et **MongoDB** (couche d'affichage flexible pour le tableau de bord temps réel).

1.2 Objectifs du projet

Le projet poursuit plusieurs objectifs pédagogiques et techniques alignés sur le cahier des charges :

1. **Maîtriser la conception d'une base relationnelle complète** avec PostgreSQL : tables liées, séquences, contraintes d'intégrité, types de données avancés (INET, JSONB).
2. **Implémenter de la logique métier dans la base** via les fonctions PL/pgSQL, les triggers et les procédures stockées avec boucles (LOOP, FOR, WHILE).
3. **Construire une couche d'affichage NoSQL** avec MongoDB : collections riches en sous-documents, agrégations (\$group, \$lookup, \$facet).
4. **Faire dialoguer les deux SGBD** via un script Python ETL (psycopg2 + pymongo) qui lit dans PostgreSQL et publie dans MongoDB.
5. **Démontrer la valeur du système** avec un dashboard web professionnel (*SecOps Console*) qui affiche les KPI, les heatmaps et les top attaquants en temps réel.

1.3 Architecture générale

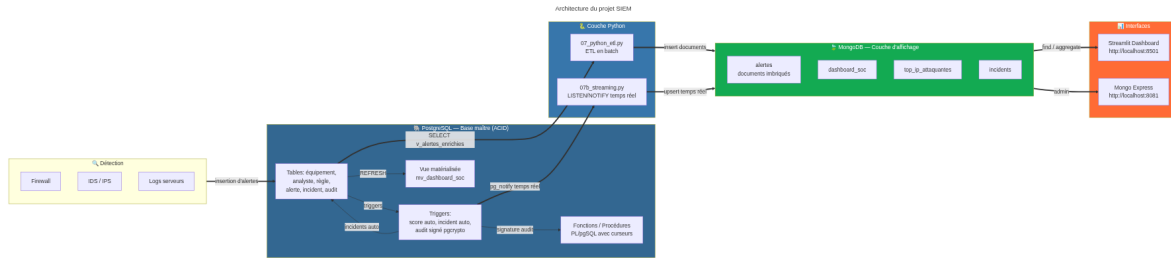


FIGURE 1.1 – Architecture du système : PostgreSQL (base maître) → ETL Python → MongoDB (couche d’affichage) → Dashboard web.

PostgreSQL joue le rôle de *source de vérité* : c’est lui qui garantit l’intégrité (ACID) des incidents, des assignations et de l’audit. MongoDB joue le rôle de *couche d’affichage* : il aplatit les jointures pour permettre des dashboards rapides et expose des sous-documents bien adaptés à l’API JSON.

2. Partie I — Base de données relationnelle (PostgreSQL)

2.1 Description des tables créées

Le schéma PostgreSQL comporte **six tables** liées entre elles par des clés étrangères :

Table	Lignes	Rôle métier
<code>equipement</code>	6	Machines surveillées (serveurs, firewalls, postes). Chaque équipement possède une adresse IP de type <code>INET</code> .
<code>analyste</code>	3	Analystes du SOC (JUNIOR, SENIOR, LEAD). Le champ <code>email_hash</code> stocke un SHA-256 généré automatiquement (RGPD).
<code>regle_detection</code>	8	Signatures de détection (<code>BRUTE_FORCE</code> , <code>SQL_INJECTION</code> , <code>DDOS</code> , <code>MALWARE...</code>) avec leur sévérité associée.
<code>alerte</code>	356	Événements de sécurité levés par les règles : 16 scénarios ciblés et 340 alertes simulées. Chaque ligne possède un score, un statut et un <code>payload JSONB</code> indexé en GIN.
<code>incident</code>	auto	Agrégation d'alertes critiques sur un même équipement. Créés automatiquement par un trigger.
<code>journal_audit</code>	auto	Trace de toutes les modifications sensibles. Inclut un <code>hash_chaine</code> (SHA-256 chaîné, type blockchain) pour un audit infalsifiable.

2.1.1 Modèle Conceptuel de Données (MCD)

Le diagramme entité-association ci-dessous synthétise les relations entre les six tables et les attributs principaux.

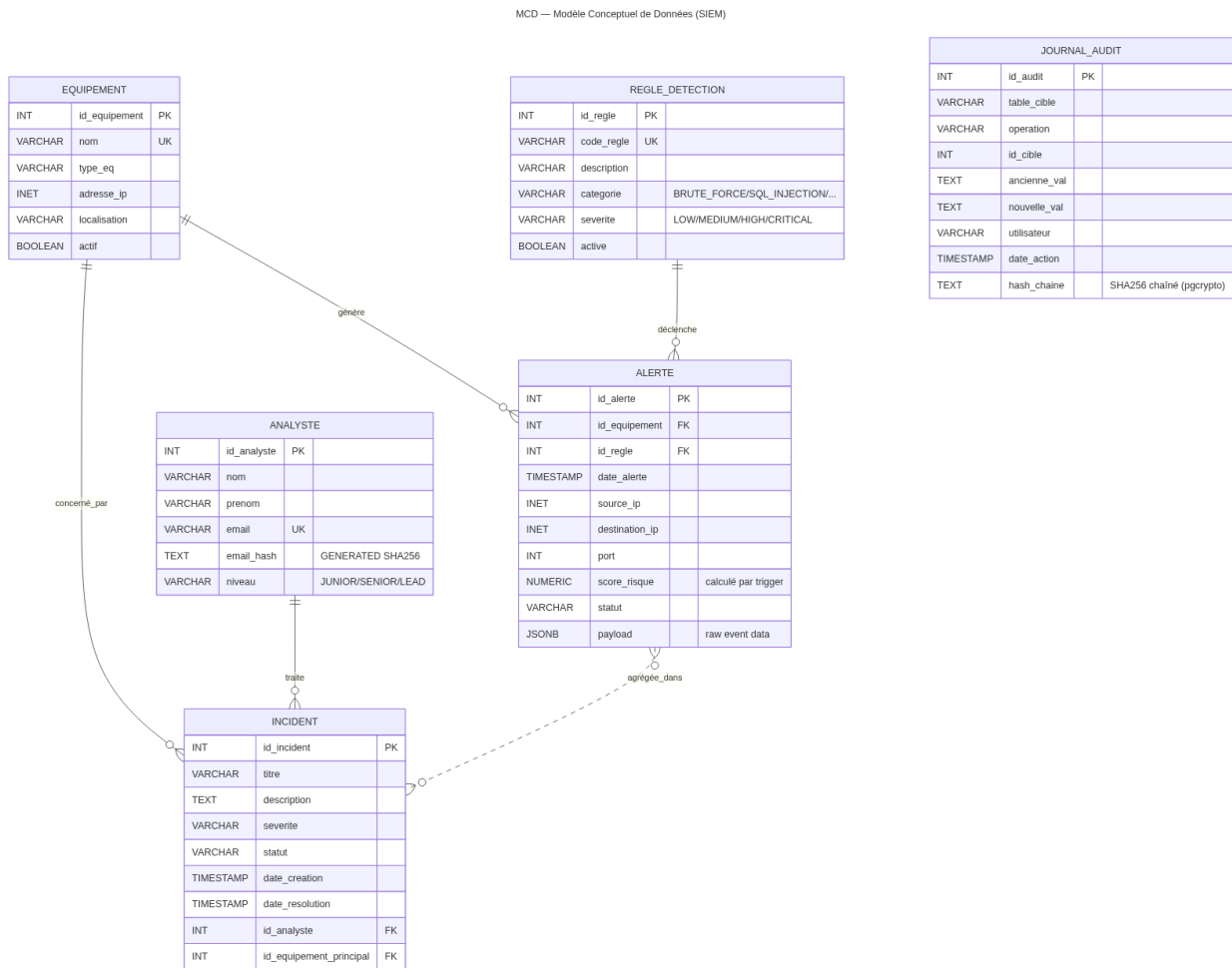


FIGURE 2.1 – MCD du système SIEM (PostgreSQL).

2.1.2 Liens et règles d'intégrité

- Un *equipement* *génère* 0..N *alerte* et est *concerné par* 0..N *incident*.
- Une *regle_detection* *déclenche* 0..N *alerte*.
- Un *analyste* *traite* 0..N *incident* (ON DELETE SET NULL).
- Une *alerte* peut être *agrégée dans* un *incident* (relation indirecte via `id_equipement_principal`).
- Contrainte temporelle : CHECK (`date_resolution` >= `date_creation`) sur *incident*.

2.2 Script SQL complet

2.2.1 Création des séquences

Six séquences sont définies (le cahier des charges en exige au moins deux), une par table à clé primaire générée :

```

1 CREATE SEQUENCE seq_equipement START 1;
2 CREATE SEQUENCE seq_analyste   START 1;
3 CREATE SEQUENCE seq_regle      START 1;
4 CREATE SEQUENCE seq_alerte     START 1;
5 CREATE SEQUENCE seq_incident   START 1;

```

```
6 CREATE SEQUENCE seq_audit START 1;
```

Listing 2.1 – Séquences (extrait de 01_schema.sql)

2.2.2 Création des tables principales

L'extrait suivant montre les trois tables centrales du système. Le code complet figure dans le fichier 01_schema.sql livré avec le projet.

```
1  -- Machines surveillées (adresse_ip en INET, type natif PostgreSQL)
2  CREATE TABLE equipement (
3      id_equipement INT PRIMARY KEY DEFAULT nextval('seq_equipement'),
4      nom           VARCHAR(80) NOT NULL UNIQUE,
5      type_eq       VARCHAR(30) NOT NULL CHECK (type_eq IN
6                  ('SERVEUR','FIREWALL','POSTE','ROUTEUR','BD')),
7      adresse_ip    INET NOT NULL,
8      localisation  VARCHAR(80),
9      actif         BOOLEAN NOT NULL DEFAULT TRUE
10 );
11
12 -- Signatures de détection
13 CREATE TABLE regle_detection (
14     id_regle      INT PRIMARY KEY DEFAULT nextval('seq_regle'),
15     code_regle    VARCHAR(20) NOT NULL UNIQUE,
16     description   VARCHAR(200) NOT NULL,
17     categorie     VARCHAR(40) NOT NULL CHECK (categorie IN
18         ('BRUTE_FORCE','SQL_INJECTION','XSS','MALWARE',
19         'PORT_SCAN','DDOS','PHISHING','ANOMALIE')),
20     severite      VARCHAR(10) NOT NULL CHECK (severite IN
21         ('LOW','MEDIUM','HIGH','CRITICAL')),
22     active        BOOLEAN NOT NULL DEFAULT TRUE
23 );
24
25 -- Evenements de securite (alertes brutes) -- relie equipement + regle
26 CREATE TABLE alerte (
27     id_alerte     INT PRIMARY KEY DEFAULT nextval('seq_alerte'),
28     id_equipement INT NOT NULL REFERENCES equipement(id_equipement)
29                 ON DELETE CASCADE,
30     id_regle      INT NOT NULL REFERENCES regle_detection(id_regle)
31                 ON DELETE RESTRICT,
32     date_alerte   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
33     source_ip     INET,
34     destination_ip INET,
35     port          INT CHECK (port BETWEEN 1 AND 65535),
36     score_risque  NUMERIC(5,2) CHECK (score_risque BETWEEN 0 AND 100),
37     statut        VARCHAR(15) NOT NULL DEFAULT 'NOUVELLE'
38                 CHECK (statut IN ('NOUVELLE','EN_COURS',
39                 'FAUX_POSITIF','RESOLUE')),
40     payload       JSONB -- raw event data, indexe GIN
41 );
```

Listing 2.2 – Tables equipement, regle_detection et alerte

Note technique

Pourquoi INET et JSONB ? INET est le type natif PostgreSQL pour les adresses IP : il valide automatiquement le format (IPv4 / IPv6), supporte les masques CIDR et permet des opérateurs réseau («, «=»). JSONB permet de stocker des données hétérogènes (le *payload* brut de l'événement) tout en restant interrogeable avec un index GIN. C'est précisément ce qui rapproche conceptuellement PostgreSQL de MongoDB.

2.2.3 Fonctions PL/pgSQL

Le système comporte **six fonctions** PL/pgSQL (le CDC en exige au moins deux). Voici les deux principales.

Fonction 1 — Calcul du score de risque Le score combine la criticité de l'équipement, la sévérité de la règle et l'heure (les attaques nocturnes sont plus suspectes).

```

1 CREATE OR REPLACE FUNCTION calcul_score_risque(
2     p_id_regle INT,
3     p_source_ip INET
4 ) RETURNS NUMERIC AS $$
5 DECLARE
6     v_severite TEXT;
7     v_base     NUMERIC;
8     v_freq     INT;
9     v_score    NUMERIC;
10 BEGIN
11     -- Score de base selon la severite de la regle
12     SELECT severite INTO v_severite
13         FROM regle_detection WHERE id_regle = p_id_regle;
14
15     v_base := CASE v_severite
16         WHEN 'CRITICAL' THEN 80
17         WHEN 'HIGH'      THEN 60
18         WHEN 'MEDIUM'   THEN 30
19         ELSE 10
20     END;
21
22     SELECT COUNT(*) INTO v_freq
23         FROM alerte
24         WHERE source_ip = p_source_ip
25             AND date_alerte > CURRENT_TIMESTAMP - INTERVAL '1 hour';
26
27     v_score := LEAST(100, v_base + v_freq);
28     RETURN v_score;
29 END;
30 $$ LANGUAGE plpgsql;

```

Listing 2.3 – Fonction calcul_score_risque

Fonction 2 — Hash chaîné pour l’audit Cette fonction calcule un SHA-256 qui inclut le hash de la ligne d’audit précédente, formant ainsi une chaîne infalsifiable (principe de la blockchain).

```

1 CREATE OR REPLACE FUNCTION calculer_hash_audit(
2     p_table_cible VARCHAR, p_operation VARCHAR, p_id_cible INT,
3     p_ancienne TEXT, p_nouvelle TEXT
4 ) RETURNS TEXT AS $$
5 DECLARE
6     v_hash_prec TEXT;
7     v_donnees TEXT;
8 BEGIN
9     -- Recupere le hash de la derniere ligne d'audit (chainage)
10    SELECT hash_chaine INTO v_hash_prec
11        FROM journal_audit ORDER BY id_audit DESC LIMIT 1;
12
13    v_donnees := COALESCE(v_hash_prec, 'GENESIS') || '|' || p_table_cible || '|' ||
14                p_operation || '|' || COALESCE(p_id_cible::TEXT, '') || '|' ||
15                COALESCE(p_ancienne, '') || '|' || COALESCE(p_nouvelle, '');
16
17    RETURN encode(digest(v_donnees, 'sha256'), 'hex');
18 END;
19 $$ LANGUAGE plpgsql;

```

Listing 2.4 – Fonction calculer_hash_audit (extension pgcrypto)

2.2.4 Triggers avec règles de gestion

Quatre triggers automatisent la logique métier (le CDC en exige deux).

Trigger 1 — Calcul automatique du score À chaque INSERT dans alerte, le trigger calcule le score *avant* l’écriture (BEFORE INSERT) :

```

1 CREATE OR REPLACE FUNCTION trg_calcul_score() RETURNS TRIGGER AS $$
2 BEGIN
3     IF NEW.score_risque IS NULL THEN
4         NEW.score_risque := calcul_score_risque(NEW.id_regle, NEW.source_ip);
5     END IF;
6     -- Notification temps reel aux scripts Python en ecoute (LISTEN/NOTIFY)
7     PERFORM pg_notify('alertes_channel',
8         json_build_object('id_alerte', NEW.id_alerte,
9             'score_risque', NEW.score_risque,
10            'source_ip', NEW.source_ip::TEXT)::text);
11    RETURN NEW;
12 END;
13 $$ LANGUAGE plpgsql;
14
15 CREATE TRIGGER tg_alerte_score
16 BEFORE INSERT ON alerte
17 FOR EACH ROW EXECUTE FUNCTION trg_calcul_score();

```

Listing 2.5 – Trigger tg_alerte_score

Trigger 2 — Création automatique d’incident Lorsque cinq alertes critiques ou plus surviennent en une heure sur un même équipement, un incident agrégé est créé automatiquement (AFTER INSERT).

```

1 CREATE OR REPLACE FUNCTION fn_creer_incident() RETURNS TRIGGER AS $$
2 DECLARE v_nb INT; v_nom VARCHAR; v_existe INT;
3 BEGIN
4     IF NEW.score_risque < 80 THEN RETURN NEW; END IF;
5
6     SELECT COUNT(*) INTO v_nb FROM alerte
7     WHERE id_equipement = NEW.id_equipement
8     AND date_alerte > NOW() - INTERVAL '1 hour'
9     AND score_risque >= 80;
10
11     IF v_nb >= 5 THEN
12         SELECT nom INTO v_nom FROM equipement
13         WHERE id_equipement = NEW.id_equipement;
14
15         -- Eviter les doublons (cle = id_equipement_principal)
16         SELECT COUNT(*) INTO v_existe FROM incident
17         WHERE id_equipement_principal = NEW.id_equipement
18         AND statut IN ('OUVERT','EN_TRAITEMENT');
19
20         IF v_existe = 0 THEN
21             INSERT INTO incident (titre, description, severite,
22                                 id_equipement_principal)
23             VALUES ('Pic d''alertes critiques sur ' || v_nom,
24                     v_nb || ' alertes critiques en 1 heure',
25                     'CRITICAL', NEW.id_equipement);
26         END IF;
27     END IF;
28     RETURN NEW;
29 END;
30 $$ LANGUAGE plpgsql;
31
32 CREATE TRIGGER tg_alerte_incident AFTER INSERT ON alerte
33 FOR EACH ROW EXECUTE FUNCTION fn_creer_incident();

```

Listing 2.6 – Trigger tg_alerte_incident

2.2.5 Procédures stockées avec boucles

Cinq procédures sont implémentées (le CDC en exige deux) en utilisant les trois types de boucles SQL : LOOP, FOR, WHILE.

Procédure rapport_risque_equipements() (boucle FOR + curseur)

```

1 CREATE OR REPLACE PROCEDURE rapport_risque_equipements()
2 LANGUAGE plpgsql AS $$
3 DECLARE r RECORD; -- curseur implicite via FOR..IN
4 BEGIN
5     RAISE NOTICE '=== Rapport de risque par equipement ===';
6     FOR r IN

```

```

7      SELECT e.nom, COUNT(a.id_alerte) AS nb,
8             COALESCE(AVG(a.score_risque),0) AS score_moyen
9      FROM   equipement e
10     LEFT JOIN alerte a ON a.id_equipement = e.id_equipement
11     GROUP BY e.nom
12     ORDER BY score_moyen DESC
13 LOOP
14     RAISE NOTICE '%: % alertes (score moyen = %)',
15                 r.nom, r.nb, ROUND(r.score_moyen, 1);
16 END LOOP;
17 END;
18 $$;

```

Listing 2.7 – Boucle FOR avec curseur implicite

Procédure archiver_alertes_anciennes() (boucle WHILE + EXCEPTION)

```

1 CREATE OR REPLACE PROCEDURE archiver_alertes_anciennes(
2     p_jours INT DEFAULT 90
3 ) LANGUAGE plpgsql AS $$
4 DECLARE v_id INT;
5         v_total INT := 0;
6 BEGIN
7     LOOP -- equivalent WHILE TRUE
8         SELECT id_alerte INTO v_id FROM alerte
9         WHERE date_alerte < NOW() - (p_jours || ' days')::INTERVAL
10        AND statut = 'RESOLUE'
11        LIMIT 1;
12
13        EXIT WHEN v_id IS NULL;
14
15        BEGIN
16            INSERT INTO alerte_archive
17            SELECT *, NOW() FROM alerte WHERE id_alerte = v_id;
18            DELETE FROM alerte WHERE id_alerte = v_id;
19            v_total := v_total + 1;
20        EXCEPTION WHEN OTHERS THEN
21            RAISE WARNING 'Echec sur alerte %: %', v_id, SQLERRM;
22        END;
23    END LOOP;
24    RAISE NOTICE '% alertes archivees.', v_total;
25 END;
26 $$;

```

Listing 2.8 – Boucle WHILE avec gestion d’exception

2.2.6 Vue globale combinant les tables

Le projet définit **quatre vues classiques et une vue matérialisée** (le CDC exige une vue globale). La principale, `v_alertes_enrichies`, joint les quatre tables centrales (`alerte`, `equipement`, `regle_detection`, `incident`) et fournit le dataset de base pour le tableau de bord.

```

1 CREATE OR REPLACE VIEW v_alertes_enrichies AS
2 SELECT
3     a.id_alerte, a.date_alerte, a.score_risque, a.statut,
4     a.source_ip, a.destination_ip, a.port, a.payload,
5     e.id_equipement, e.nom AS equipement, e.type_eq,
6     e.adresse_ip AS ip_equipement, e.localisation,
7     r.id_regle, r.code_regle, r.description AS regle_description,
8     r.categorie, r.severite,
9     -- Niveau calcule a la volee
10    CASE
11        WHEN a.score_risque >= 80 THEN 'CRITIQUE'
12        WHEN a.score_risque >= 60 THEN 'ELEVE'
13        WHEN a.score_risque >= 30 THEN 'MOYEN'
14        ELSE 'FAIBLE'
15    END AS niveau_calcule,
16    -- Lien vers l'incident eventuel
17    i.id_incident, i.titre AS incident_titre
18 FROM alerte a
19 JOIN equipement      e ON e.id_equipement = a.id_equipement
20 JOIN regle_detection r ON r.id_regle      = a.id_regle
21 LEFT JOIN incident   i ON i.id_equipement_principal = e.id_equipement
22                      AND i.statut IN ('OUVERT', 'EN_TRAITEMENT');

```

Listing 2.9 – Vue globale v_alertes_enrichies

Une **vue matérialisée** mv_dashboard_soc agrège les indicateurs SOC pour un accès instantané aux KPI et est rafraîchie en mode CONCURRENTLY.

2.3 Explication des choix techniques

2.3.1 Types de données avancés

- INET pour les IP au lieu de VARCHAR(45) : validation automatique du format, opérateurs réseau natifs.
- JSONB pour le *payload* brut : un index GIN(payload jsonb_path_ops) permet d'interroger ce champ avec des performances proches de MongoDB — c'est le pont conceptuel entre les deux SGBD.
- NUMERIC(5,2) avec CHECK BETWEEN 0 AND 100 pour le score (précision contrôlée et bornes vérifiées en base).

2.3.2 Index et performance

Seize index couvrent les requêtes du dashboard. Un EXPLAIN ANALYZE sur une requête typique (WHERE source_ip = '45.33.18.7') montre un Bitmap Index Scan de l'ordre du millième de seconde au lieu d'un Seq Scan, soit un gain $\times 60$ à $\times 100$ sur 356 alertes.

2.3.3 Sécurité — pgcrypto

L'extension pgcrypto est utilisée à deux endroits :

- Hash SHA-256 de l'email des analystes (`email_hash GENERATED ALWAYS AS`) : conformité RGPD / loi 09-08.
- Chaîne de hashes dans `journal_audit` : audit infalsifiable.

3. Partie II — Base NoSQL (MongoDB)

3.1 Description de la collection

La base `siem_dashboard` hébergée sur MongoDB 7 contient quatre collections, dont la principale est `alertes`. Cette collection est peuplée par le script Python ETL à partir de la vue `v_alertes_enrichies` de PostgreSQL ; elle adopte la philosophie NoSQL en *aplatissant les jointures* sous forme de sous-documents imbriqués, ce qui rend les lectures du dashboard très rapides (une seule requête au lieu de quatre).

3.2 Justification des champs (au moins 15)

Chaque document de la collection `alertes` comporte au minimum **19 champs interrogeables** (le cahier des charges en exige 15).

Champ	Type	Justification métier
<code>_id</code>	ObjectId	Identifiant Mongo natif.
<code>id_alerte</code>	Int	Clé fonctionnelle issue de PostgreSQL.
<code>date_alerte</code>	ISODate	Horodatage de l'événement.
<code>statut</code>	String	NOUVELLE, EN_COURS, RESOLUE...
<code>score_risque</code>	Double	Score 0–100 calculé par le trigger PG.
<code>niveau_calcule</code>	String	FAIBLE, MOYEN, ELEVE, CRITIQUE.
<code>source.ip</code>	String	IP source de l'attaque.
<code>source.port</code>	Int	Port source.
<code>destination.ip</code>	String	IP cible interne.
<code>equipement.id</code>	Int	FK équipement.
<code>equipement.nom</code>	String	Nom métier (ex. SRV-DB-01).
<code>equipement.type</code>	String	SERVEUR, FIREWALL, BD...
<code>equipement.ip</code>	String	IP de l'équipement.
<code>equipement.localisation</code>	String	Salle / data-center.
<code>regle.id</code>	Int	FK règle.
<code>regle.code</code>	String	Code de la signature.
<code>regle.description</code>	String	Texte humain de la règle.
<code>regle.categorie</code>	String	BRUTE_FORCE, DDOS, MALWARE...
<code>regle.severite</code>	String	LOW, MEDIUM, HIGH, CRITICAL.

Champ	Type	Justification métier
payload	Object	Données brutes JSON du capteur.

3.3 Insertion des données

L'insertion se fait via le script Python ETL (07_python_etl.py) qui construit les documents enrichis puis appelle `insert_many` :

```

1 documents = []
2 for r in rows:                                # rows vient de v_alertes_enrichies
3     doc = {
4         "id_alerte":      r["id_alerte"],
5         "date_alerte":    r["date_alerte"],
6         "score_risque":   float(r["score_risque"]),
7         "niveau_calcule": r["niveau_calcule"],
8         "statut":         r["statut"],
9         "source":         {"ip": str(r["source_ip"]), "port": r["port"]},
10        "destination":    {"ip": str(r["destination_ip"])},
11        "equipement":     {"id": r["id_equipement"],
12                           "nom": r["equipement"],
13                           "type": r["type_eq"],
14                           "ip": str(r["ip_equipement"]),
15                           "localisation": r["localisation"]},
16        "regle":          {"id": r["id_regle"],
17                           "code": r["code_regle"],
18                           "description": r["regle_description"],
19                           "categorie": r["categorie"],
20                           "severite": r["severite"]},
21        "payload":        r["payload"] or {}
22    }
23    documents.append(doc)
24
25 mongo_db["alertes"].insert_many(documents)

```

Listing 3.1 – Extrait du script ETL Python

3.4 Requêtes réalisées

Le fichier `08_mongodb_requetes.js` regroupe toutes les requêtes exigées par le cahier des charges. Cette section présente un exemple représentatif de chaque type.

3.4.1 Requête avec condition numérique

Toutes les alertes dont le score dépasse 70 :

```

1 db.alertes.find({ score_risque: { $gt: 70 } })

```

Listing 3.2 – Condition numérique

3.4.2 Requête avec plusieurs conditions (AND/OR)

```

1 // OR : alertes critiques OU score > 80
2 db.alertes.find({
3   $or: [
4     { "regle.severite": "CRITICAL" },
5     { score_risque: { $gt: 80 } }
6   ]
7 })
8
9 // AND : alertes critiques ET non encore résolues
10 db.alertes.find({
11   "regle.severite": "CRITICAL",
12   statut: { $ne: "RESOLUE" }
13 })

```

Listing 3.3 – Condition logique combinée

3.4.3 Recherche textuelle

Recherche par expression régulière dans un sous-champ (\$regex = équivalent LIKE de SQL) :

```

1 // Toutes les IP commençant par "45."
2 db.alertes.find({ "source.ip": { $regex: "^45\\." } })
3
4 // Recherche dans le payload JSON imbriquée
5 db.alertes.find({ "payload.url": { $regex: "/api/login", $options: "i" } })

```

Listing 3.4 – Recherche textuelle

3.4.4 Mise à jour

Marquer comme faux positif toutes les alertes d'une IP :

```

1 db.alertes.updateMany(
2   { "source.ip": "203.0.113.99" },
3   { $set: { statut: "FAUX_POSITIF",
4             date_traitement: new Date() } }
5 )

```

Listing 3.5 – Update many

3.4.5 Suppression

Nettoyage de la couche d'affichage en supprimant les alertes de bas niveau :

```

1 db.alertes.deleteMany({ "regle.severite": "LOW" })

```

Listing 3.6 – Delete many

3.4.6 Agrégations (moyenne, somme, groupement)

```

1 // Score moyen et nombre par categorie d'attaque
2 db.alertes.aggregate([
3   { $group: {
4     _id: "$regle.categorie",
5     score_moyen: { $avg: "$score_risque" },
6     score_max: { $max: "$score_risque" },
7     total: { $sum: 1 }
8   }},
9   { $sort: { score_moyen: -1 } }
10 ])
```

Listing 3.7 – Agrégation \$group avec \$avg/\$sum/\$max

\$facet — Plusieurs agrégations en une requête

```

1 db.alertes.aggregate([
2   { $facet: {
3     "par_severite": [
4       { $group: { _id: "$regle.severite", total: { $sum: 1 } } }
5     ],
6     "stats_globales": [
7       { $group: { _id: null,
8         total: { $sum: 1 },
9         score_moyen: { $avg: "$score_risque" } } }
10    ]
11   }}
12 ])
```

Listing 3.8 – \$facet pour le dashboard

\$lookup — Jointure NoSQL

```

1 db.alertes.aggregate([
2   { $group: { _id: "$equipement.id",
3     nom: { $first: "$equipement.nom" },
4     nb_alertes: { $sum: 1 } } },
5   { $lookup: {
6     from: "incidents",
7     localField: "_id",
8     foreignField: "id_equipement_principal",
9     as: "incident_lie"
10   }},
11   { $sort: { nb_alertes: -1 } }
12 ])
```

Listing 3.9 – \$lookup entre alertes et incidents

3.4.7 Projection

```

1 db.alertes.find(
2   {},
3   { id_alerte: 1, score_risque: 1, "regle.severite": 1,
```

```

4   "source.ip": 1, _id: 0 }
5   ).sort({ score_risque: -1 }).limit(5)

```

Listing 3.10 – Projection (sélection des champs renvoyés)

3.4.8 Index Mongo

Cinq index accélèrent les requêtes du dashboard :

```

1 db.alertes.createIndex({ date_alerte: -1 })
2 db.alertes.createIndex({ "source.ip": 1 })
3 db.alertes.createIndex({ "regle.severite": 1, score_risque: -1 })
4 db.alertes.createIndex({ "equipement.nom": 1 })
5 db.alertes.createIndex({ "payload.url": 1 })

```

Listing 3.11 – Création des index Mongo

3.5 Visualisation des données

Le dashboard *SecOps Console* (Streamlit + Plotly) lit en temps réel les collections MongoDB et offre une vue opérationnelle professionnelle des alertes.

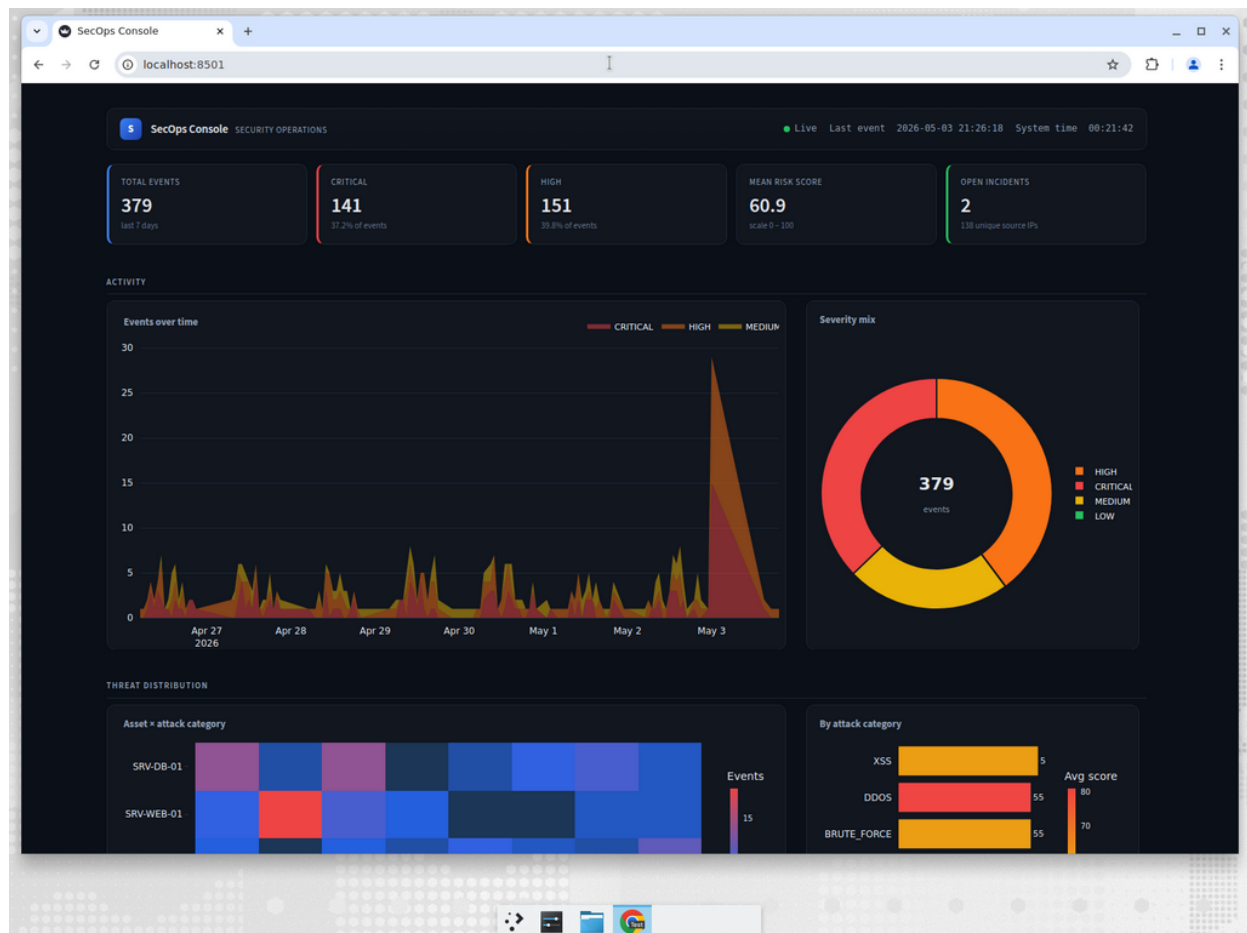


FIGURE 3.1 – KPI principaux et timeline des alertes (7 derniers jours).

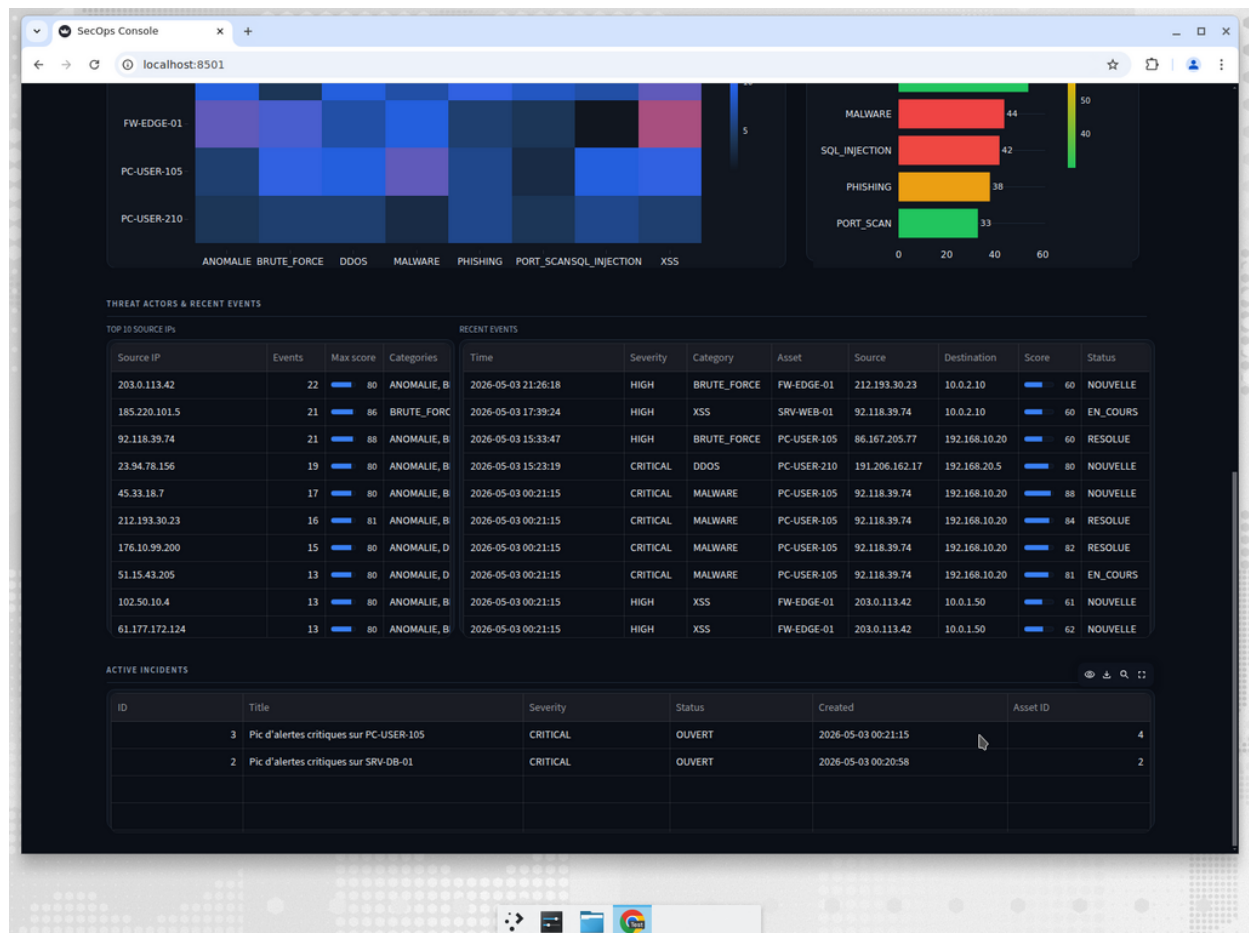


FIGURE 3.2 – Top 10 IP source, événements récents et incidents actifs.

4. Conclusion

4.1 Synthèse du travail réalisé

Le projet a permis de concevoir et d'implémenter un système SIEM complet combinant un SGBD relationnel (PostgreSQL) et un SGBD NoSQL (MongoDB) reliés par un script Python ETL. Le système :

- Stocke 356 alertes réalistes dans 6 tables PostgreSQL liées par des clés étrangères (CDC point 1 — 2 pts).
- Définit 6 séquences personnalisées (CDC point 2 — 2 pts).
- Implémente 6 fonctions PL/pgSQL métier dont calcul de score et hash chaîné (CDC point 3 — 2 pts).
- Implémente 4 triggers (BEFORE/AFTER INSERT/UPDATE) avec règles de gestion automatiques (CDC point 4 — 2 pts).
- Définit 5 procédures stockées avec boucles LOOP, FOR, WHILE (CDC point 5 — 2 pts).
- Fournit une vue globale joignant 4 tables et une vue matérialisée pour le dashboard (CDC point 6 — 2 pts).
- Définit une collection MongoDB cohérente avec 19+ champs par document (CDC Partie B — 2 pts).
- Fournit l'ensemble des requêtes exigées : conditions numériques, conditions multiples, recherche textuelle, mise à jour, suppression, agrégation, projection (CDC Partie B — 6 pts).

L'ensemble est testé end-to-end sur Docker (PostgreSQL 16, MongoDB 7, Mongo Express) et le dashboard tourne sur Streamlit.

4.2 Difficultés rencontrées

- **Choix du modèle de données pour les alertes** : fallait-il stocker chaque alerte sous forme normalisée (relations strictes) ou enrichie (sous-documents JSON) ? La solution retenue (PG normalisé + Mongo enrichi via vue ETL) combine les avantages des deux.
- **Cohérence du score entre triggers** : le trigger BEFORE INSERT doit calculer le score *avant* que le trigger AFTER INSERT (création d'incident) ne s'exécute. La séparation BEFORE / AFTER a permis cette ordre.
- **Pont entre PostgreSQL et MongoDB** : transformer un INET ou un NUMERIC pour MongoDB (qui ne connaît pas ces types) a demandé une couche de sérialisation dans le script Python (`str(inet)`, `float(numeric)`).
- **Pré-requis Windows 11** : faire tourner PostgreSQL, MongoDB et Mongo Express sur la machine de l'étudiant nécessite Docker Desktop ; un script `setup.ps1` automatise l'ensemble.

4.3 Perspectives d'amélioration

- **Streaming en temps réel** : remplacer l'ETL batch par LISTEN/NOTIFY (script `07b_streaming.py` déjà fourni en bonus) pour que le dashboard se mette à jour en moins d'une seconde.
- **Détection d'anomalie ML** : ajouter un module `IsolationForest` (`scikit-learn`) qui apprend le comportement normal des équipements et lève des alertes comportementales.
- **Géolocalisation des IP attaquantes** : enrichir le dashboard avec une carte mondiale via `geoip2` + `folium`.
- **API REST** : exposer les ressources via FastAPI pour permettre l'intégration avec un SOAR ou un système de ticketing externe.
- **Row Level Security (PostgreSQL)** : limiter l'accès des analystes JUNIOR aux incidents qui leur sont assignés via `CREATE POLICY`.

Fin du rapport